

CHAPTER 6

RECURSION

IN THIS CHAPTER, YOU WILL:

- Learn about recursive definitions
- Explore the base case and the general case of a recursive definition
- Learn about recursive algorithm
- Learn about recursive functions
- Explore how to use recursive functions to implement recursive algorithms

In previous chapters, to devise solutions to problems we used the most common technique, called iteration. For certain problems, however, using the iterative technique to obtain the solution is quite complicated. This chapter introduces another problem-solving technique, called recursion, and provides several examples demonstrating how recursion works.

Recursive Definitions

The process of solving a problem by reducing it to smaller versions of itself is called **recursion**. Recursion is a very powerful way to solve certain problems for which the solution would otherwise be very complicated. Let us consider a problem that is familiar to most everyone.

In an algebra course, you probably learned how to find the factorial of a nonnegative integer. For example, the factorial of 5, written 5!, is $5 \times 4 \times 3 \times 2 \times 1 = 120$. Similarly, $4! = 4 \times 3 \times 2 \times 1 = 24$. Also, factorial of 0 is defined to be $0! = 1$. Note that $5! = 5 \times 4 \times 3 \times 2 \times 1 = 5 \times (4 \times 3 \times 2 \times 1) = 5 \times 4!$. In general, if n is a nonnegative, the factorial of n, written as $n!$ can be defined as follows:

$$0! = 1 \qquad \text{(Equation 6-1)}$$

$$n! = n \times (n - 1)! \quad \text{if} \quad n > 0 \qquad \text{(Equation 6-2)}$$

In this definition, 0! is defined to be 1, and if n is an integer greater than 0, first we find $(n - 1)!$ and then multiply it by n. To find $(n - 1)!$, we apply the definition again. If $(n - 1) > 0$, then we use Equation 6-2; otherwise, we use Equation 6-1. Thus, for an integer n greater than 0, $n!$ is obtained by first finding $(n - 1)!$ and then multiplying $(n - 1)!$ by n.

Let us apply this definition to find 3!. Here $n = 3$. Because $n > 0$, we use Equation 6-2 to obtain:

$$3! = 3 \times 2!$$

Next, we find 2! Here $n = 2$. Because $n > 0$, we use Equation 6-2 to obtain:

$$2! = 2 \times 1!$$

Now to find 1!, we again use Equation 6-2 because $n = 1 > 0$. Thus:

$$1! = 1 \times 0!$$

Finally, we use Equation 6-1 to find 0!, which is 1. Substituting 0! into 1! gives $1! = 1$. This gives $2! = 2 \times 1! = 2 \times 1 = 2$, which in turn gives $3! = 3 \times 2! = 3 \times 2 = 6$.

The solution in Equation 6-1 is direct—that is, the right side of the equation contains no factorial notation. The solution in Equation 6-2 is given in terms of a *smaller version of itself*. The definition of the factorial given in Equations 6-1 and 6-2 is called a **recursive definition**. Equation 6-1 is called the **base case** (that is, the case for which the solution is obtained directly); Equation 6-2 is called the **general case**.

Recursive definition: A definition in which something is defined in terms of a smaller version of itself.

From the previous example (factorial), it is clear that:

1. Every recursive definition must have one (or more) base cases.
2. The general case must eventually be reduced to a base case.
3. The base case stops the recursion.

The concept of recursion in computer science works similarly. Here, we talk about recursive algorithms and recursive functions. An algorithm that finds the solution to a given problem by reducing the problem to smaller versions of itself is called a **recursive algorithm**. The recursive algorithm must have one or more base cases, and the general solution must eventually be reduced to a base case.

A function that calls itself is called a **recursive function**. That is, the body of the recursive function contains a statement that causes the same function to execute again before completing the current call. Recursive algorithms are implemented using recursive functions.

Next, let us write the recursive function that implements the factorial function.

```
int fact(int num)
{
    if (num == 0)
        return 1;
    else
        return num * fact(num - 1);
}
```

Figure 6-1 traces the execution of the following statement:

```
cout << fact(3) << endl;
```

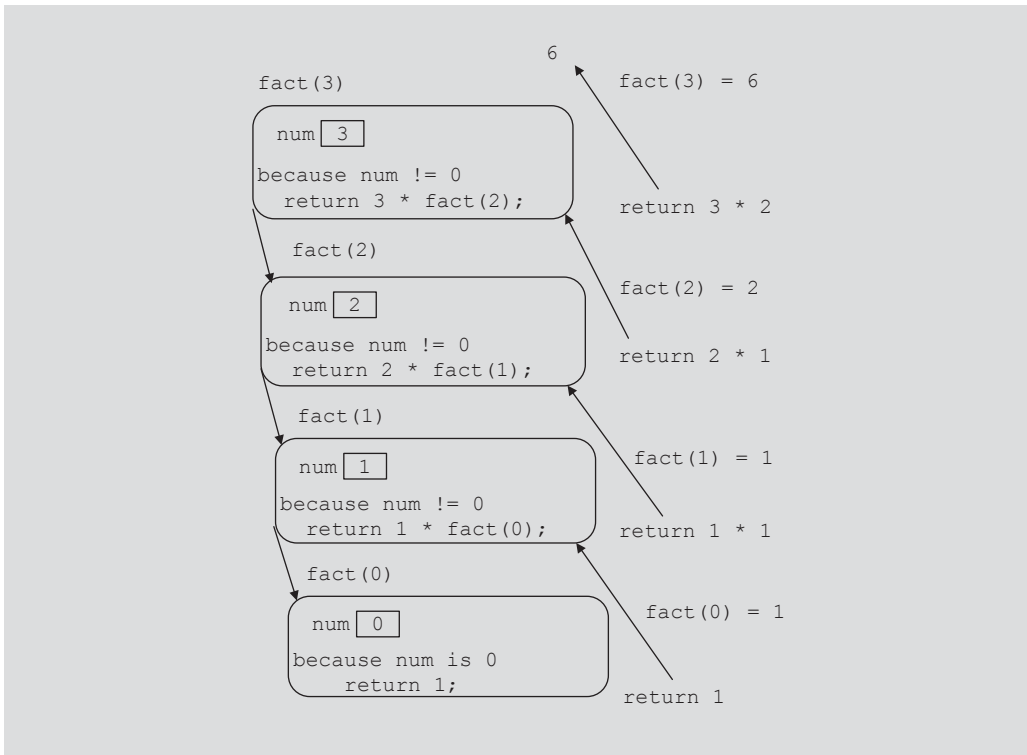


FIGURE 6-1 Execution of `fact(4)`

The output of the previous `cout` statement is: `6`

In Figure 6-1, the down arrow represents the successive calls to the function `fact`, and the upward arrows represent the values returned to the caller, that is, the calling function.

Let us note the following from the previous example, involving the factorial function:

- Logically, you can think of a recursive function as having an unlimited number of copies of itself.
- Every call to a recursive function—that is, every recursive call—has its own code and its own set of parameters and local variables.
- After completing a particular recursive call, control goes back to the calling environment, which is the previous call. The current (recursive) call must execute completely before control goes back to the previous call. The execution in the previous call begins from the point immediately following the recursive call.

Direct and Indirect Recursion

A function is called **directly recursive** if it calls itself. A function that calls another function and eventually results in the original function call is said to be **indirectly**

recursive. For example, if a function `A` calls a function `B` and function `B` calls function `A`, then function `A` is indirectly recursive. Indirect recursion can be several layers deep. For example, suppose that function `A` calls function `B`, function `B` calls function `C`, function `C` calls function `D`, and function `D` calls function `A`. Function A is then indirectly recursive.

Indirect recursion requires the same careful analysis as direct recursion. The base cases must be identified and appropriate solutions to them must be provided. However, tracing through indirect recursion can be tedious. You must, therefore, exercise extra care when designing indirect recursive functions. For simplicity, the problems in this book involve only direct recursion.

A recursive function in which the last statement executed is the recursive call is called a **tail recursive function**. The function `fact` is an example of a tail recursive function.

Infinite Recursion

Figure 6-1 shows that the sequence of recursive calls eventually reached a call that made no further recursive calls. That is, the sequence of recursive calls eventually reached a base case. On the other hand, if every recursive call results in another recursive call, the recursive function (algorithm) is said to have infinite recursion. In theory, infinite recursion executes forever. Every call to a recursive function requires the system to allocate memory for the local variables and formal parameters. The system also saves this information so that after completing a call, control can be transferred back to the right caller. Therefore, because computer memory is finite, if you execute an infinite recursive function on a computer, the function executes until the system runs out of memory and results in an abnormal termination of the program.

Recursive functions (algorithms) must be carefully designed and analyzed. You must make sure that every recursive call eventually reduces to a base case. This chapter provides several examples that illustrate how to design and implement recursive algorithms.

To design a recursive function, you must do the following:

1. Understand the problem requirements.
2. Determine the limiting conditions. For example, for a list, the limiting condition is the number of elements in the list.
3. Identify the base cases and provide a direct solution to each base case.
4. Identify the general cases and provide a solution to each general case in terms of smaller versions of itself.

Problem Solving Using Recursion

The next few sections illustrate how recursive algorithms are developed and implemented in C++ using recursive functions.

Largest Element in an Array

In this example, we use a recursive algorithm to find the largest element in an array. Consider the list given in Figure 6-2.

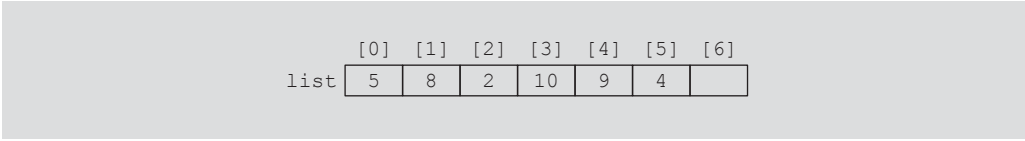


FIGURE 6-2 `list` with six elements

The largest element in the list in Figure 6-2 is `10`.

Suppose `list` is the name of the array containing the list elements. Also, suppose that `list[a]...list[b]` stands for the array elements `list[a]`, `list[a + 1]`, `...`, `list[b]`. For example, `list[0]...list[5]` represents the array elements `list[0]`, `list[1]`, `list[2]`, `list[3]`, `list[4]`, and `list[5]`. Similarly, `list[1]...list[5]` represents the array elements `list[1]`, `list[2]`, `list[3]`, `list[4]`, and `list[5]`. To write a recursive algorithm to find the largest element in `list`, let us think in terms of recursion.

If `list` is of length `1`, then `list` has only one element, which is the largest element. Suppose the length of `list` is greater than `1`. To find the largest element in `list[a]...list[b]`, we first find the largest element in `list[a + 1]...list[b]` and then compare this largest element with `list[a]`. That is, the largest element in `list[a]...list[b]` is given by:

```
maximum(list[a], largest(list[a + 1]...list[b]))
```

Let us apply this formula to find the largest element in the list shown in Figure 6-2. This list has six elements, given by `list[0]...list[5]`. Now the largest element in `list` is given by:

```
maximum(list[0], largest(list[1]...list[5]))
```

That is, the largest element in `list` is the maximum of `list[0]` and the largest element in `list[1]...list[5]`. To find the largest element in `list[1]...list[5]`, we use the same formula again because the length of this list is greater than `1`. The largest element in `list[1]...list[5]` is then:

```
maximum(list[1], largest(list[2]...list[5]))
```

and so on. We see that every time we use the previous formula to find the largest element in a sublist, the length of the sublist in the next call is reduced by one. Eventually, the sublist is of length `1`, in which case the sublist contains only one element, which is the largest element in the sublist. From this point onward, we backtrack through the recursive calls. This discussion translates into the following recursive algorithm, which is presented in pseudocode:

Base Case: The size of the list is 1
The only element in the list is the largest element

General Case: The size of the list is greater than 1
To find the largest element in list[a]...list[b]

1. Find the largest element in list[a + 1]...list[b]
   and call it max
2. Compare the elements list[a] and max
   if (list[a] >= max)
      the largest element in list[a]...list[b] is list[a]
   otherwise
      the largest element in list[a]...list[b] is max

This algorithm translates into the following C++ function to find the largest element in an array:

```
int largest(const int list[], int lowerIndex, int upperIndex)
{
    int max;

    if (lowerIndex == upperIndex) //size of the sublist is one
        return list[lowerIndex];
    else
    {
        max = largest(list, lowerIndex + 1, upperIndex);

        if (list[lowerIndex] >= max)
            return list[lowerIndex];
        else
            return max;
    }
}
```

Consider the `list` given in Figure 6-3.

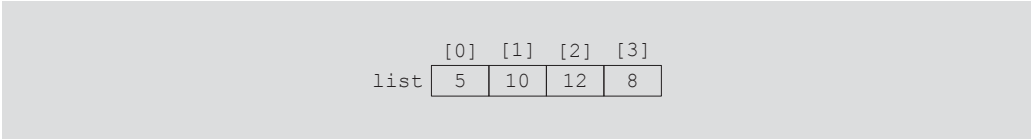


FIGURE 6-3 `list` with four elements

Let us trace the execution of the following statement:

```
cout << largest(list, 0, 3) << endl;
```

Here `upperIndex = 3` and the list has four elements. Figure 6-4 traces the execution of `largest(list, 0, 3)`.

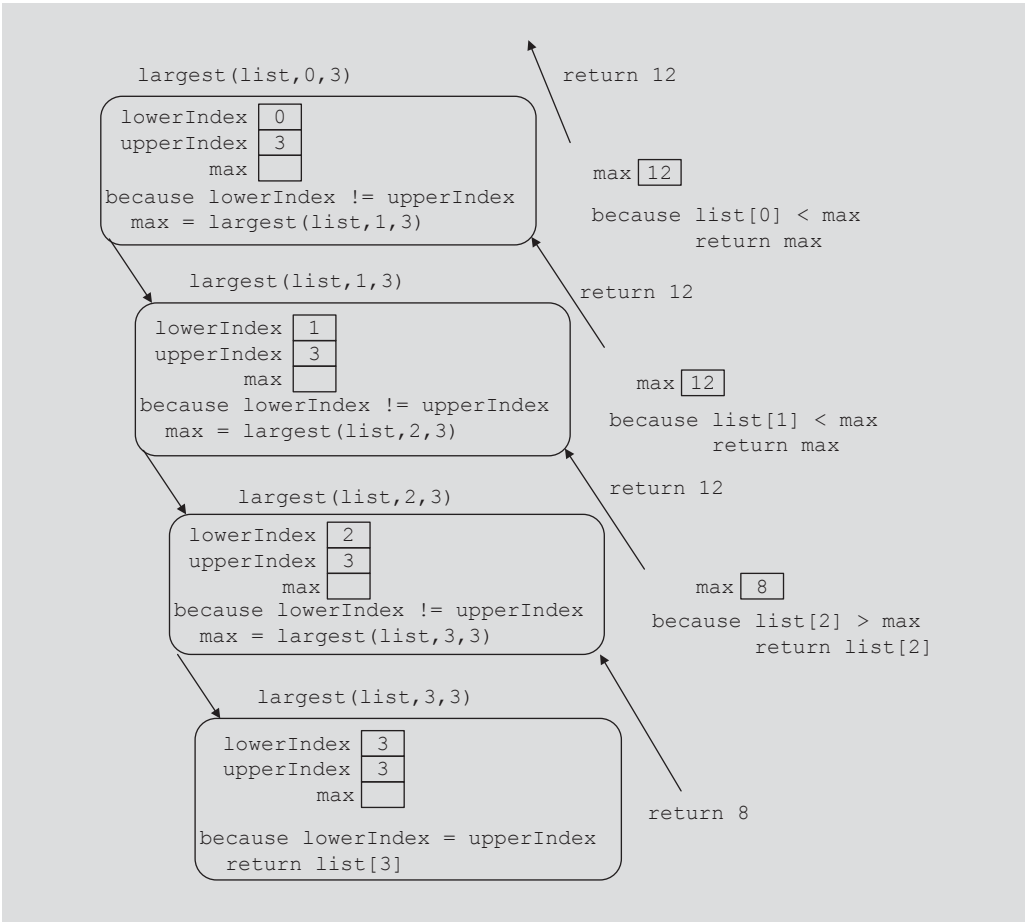


FIGURE 6-4 Execution of `largest(list, 0, 3)`

The value returned by the expression `largest(list, 0, 3)` is `12`, which is the largest element in `list`.

The following C++ program uses the function `largest` to determine the largest element in a list:

```
//***********************************************************
// Author: D.S. Malik
//
// This program uses a recursive function to find the largest
// element in a list.
//***********************************************************

#include <iostream>

using namespace std;
```

```
int largest(const int list[], int lowerIndex, int upperIndex);

int main()
{
    int intArray[10] = {23, 43, 35, 38, 67, 12, 76, 10, 34, 8};

    cout << "The largest element in intArray: "
         << largest(intArray, 0, 9);
    cout << endl;

    return 0;
}

int largest(const int list[], int lowerIndex, int upperIndex)
{
    int max;

    if (lowerIndex == upperIndex) //size of the sublist is one
        return list[lowerIndex];
    else
    {
        max = largest(list, lowerIndex + 1, upperIndex);

        if (list[lowerIndex] >= max)
            return list[lowerIndex];
        else
            return max;
    }
}
```

Sample Run:

```
The largest element in intArray: 76
```

Print a Linked List in Reverse Order

The nodes of an ordered linked list (as constructed in Chapter 5) are in ascending order. Certain applications, however, might require the data to be printed in descending order, which means that we must print the list backward. We now discuss the function **`reversePrint`**. Given a pointer to a list, this function prints the elements of the list in reverse order.

Consider the linked list shown in Figure 6-5.

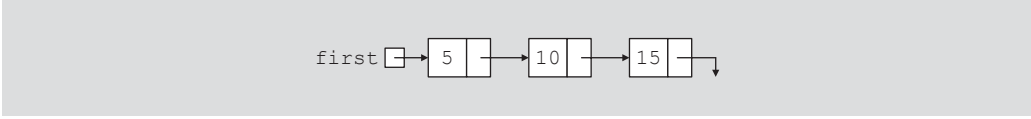


FIGURE 6-5 Linked list

For the list in Figure 6-5, the output should be in the following form:

```
15 10 5
```

Because the links are in only one direction, we cannot traverse the list backward starting from the last node. Let us see how we can effectively use recursion to print the list in reverse order.

Let us think in terms of recursion. We cannot print the `info` of the first node until we have printed the remainder of the list (that is, the tail of the first node). Similarly, we cannot print the `info` of the second node until we have printed the tail of the second node, and so on. Every time we consider the tail of a node, we reduce the size of the list by 1. Eventually, the size of the list is reduced to zero, in which case the recursion stops.

Base Case: List is empty: no action

General Case: List is nonempty

1. Print the tail
2. Print the element

Let us write this algorithm. (Suppose that `current` is a pointer to a linked list.)

```
if (current != NULL)
{
   reversePrint(current->link);      //print the tail
   cout << current->info << endl;    //print the node
}
```

Here, we do not see the base case; it is hidden. The list is printed only if the pointer, `current`, to the list is not `NULL`. Also, inside the `if` statement the recursive call is on the tail of the list. Because eventually the tail of a list will be empty, the `if` statement in the next call fails and the recursion stops. Also, note that statements (for example, printing the `info` of the node) appear after the recursive call; thus, when the transfer comes back to the calling function, we must execute the remaining statements. Recall that the function exits only after the last statement executes. (By the "last statement," we do not mean the physical last statement, but rather the logical last statement.)

Let us write a function template to implement the previous algorithm and then apply it to a list.

```
template <class Type>
void linkedListType<Type>::reversePrint
                           (nodeType<Type> *current) const
{
    if (current != NULL)
    {
        reversePrint(current->link);    //print the tail
        cout << current->info << " ";   //print the node
    }
}
```

Consider the statement

```
reversePrint(first);
```

where `first` is a pointer of type `nodeType<Type>`.

Let us trace the execution of this statement; which is a function call, for the list shown in Figure 6-5. Because the formal parameter is a value parameter, the value of the actual parameter is passed to the formal parameter. See Figure 6-6.

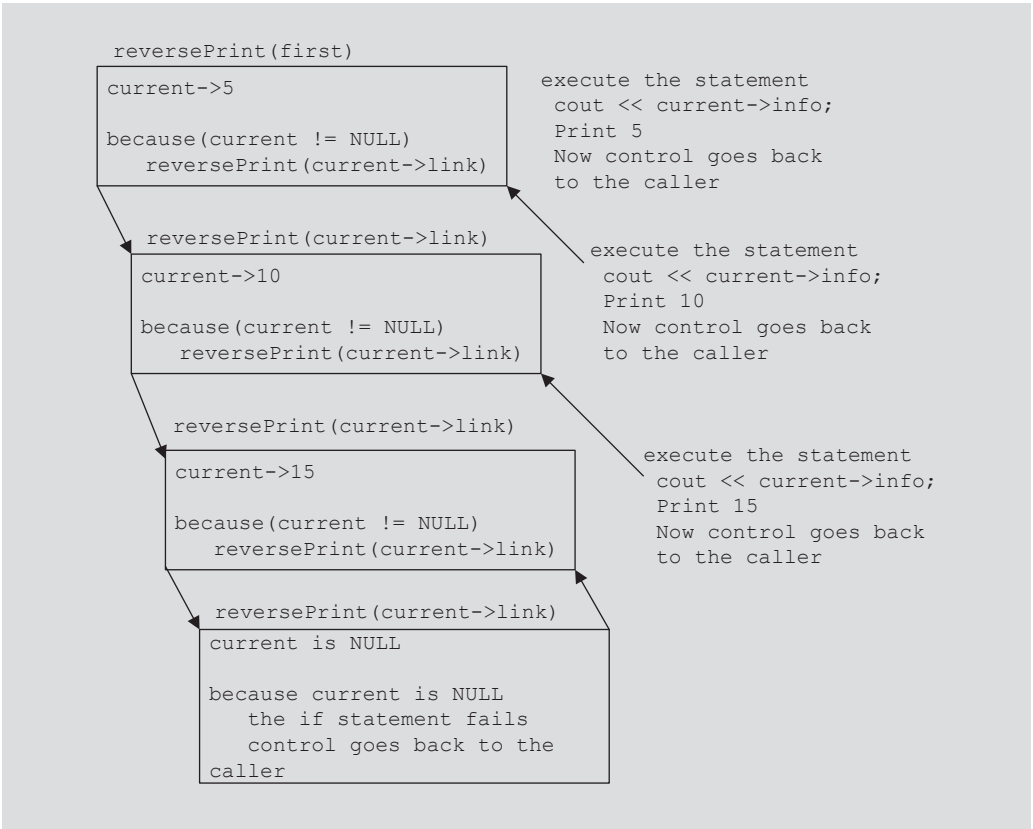


FIGURE 6-6 Execution of the statement `reversePrint(first);`

THE FUNCTION `printListReverse`

Now that we have written the function **`reversePrint`**, we can write the definition of the function **`printListReverse`**, which can be used to print an ordered linked list contained in an object of the type **`linkedListType`**. Its definition is as follows:

```
template <class Type>
void linkedListType<Type>::printListReverse() const
{
    reversePrint(first);
    cout << endl;
}
```

We can include the function **`printListReverse`** as a **`public`** member in the definition of the class and the function **`reversePrint`** as a **`private`** member. We include the

function `reversePrint` as a `private` member because it is used only to implement the function `printListReverse`.

Fibonacci Number

Consider the following sequence of numbers:

$1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots.$

Given the first two numbers of the sequence (say a_1 and a_2), the nth number a_n, $n >= 3$, of this sequence is given by:

$a_n = a_{n-1} + a_{n-2}$

Thus:

$a_3 = a_2 + a_1 = 1 + 1 = 2$, $a_4 = a_3 + a_2 = 2 + 1 = 3$, and so on.

Such a sequence is called a **Fibonacci sequence**. In the previous sequence, $a_2 = 1$ and $a_1 = 1$. However, given any first two numbers, using this process, you can determine the nth number, a_n, $n >= 3$, of such a sequence. The number determined this way is called the **nth Fibonacci number**. Suppose $a_2 = 6$ and $a_1 = 3$.

Then:

$a_3 = a_2 + a_1 = 6 + 3 = 9;$ $\qquad$ $a_4 = a_3 + a_2 = 9 + 6 = 15.$

In this example, we write a recursive function, `rFibNum`, to determine the desired Fibonacci number. The function `rFibNum` takes as parameters three numbers representing the first two numbers of the Fibonacci sequence and a number n, the desired nth Fibonacci number. The function `rFibNum` returns the nth Fibonacci number in the sequence.

Recall that the third Fibonacci number is the sum of the first two Fibonacci numbers. The fourth Fibonacci number in a sequence is the sum of the second and third Fibonacci numbers. Therefore, to calculate the fourth Fibonacci number, we add the second Fibonacci number and the third Fibonacci number (which is itself the sum of the first two Fibonacci numbers). The following recursive algorithm calculates the nth Fibonacci number, where a denotes the first Fibonacci number, b the second Fibonacci number, and n the nth Fibonacci number:

$$rFibNum(a, b, n) = \begin{cases} a & \text{if } n = 1 \\ b & \text{if } n = 2 \\ rFibNum(a, b, n-1) + & \\ rFibNum(a, b, n-2) & \text{if } n > 2. \end{cases} \quad \text{(Equation 6-3)}$$

The following recursive function implements this algorithm:

```
int rFibNum(int a, int b, int n)
{
    if (n == 1)
        return a;
    else if (n == 2)
        return b;
    else
        return rFibNum(a, b, n - 1) + rFibNum(a, b, n - 2);
}
```

Let us trace the execution of the following statement:

```
cout << rFibNum(2, 3, 4) << endl;
```

In this statement, the first number is `2`, the second number is `3`, and we want to determine the `4th` Fibonacci number of the sequence. Figure 6-7 traces the execution of the expression `rFibNum(2,3,4)`. The value returned is `8`, which is the `4th` Fibonacci number of the sequence whose first number is `2` and whose second number is `3`.

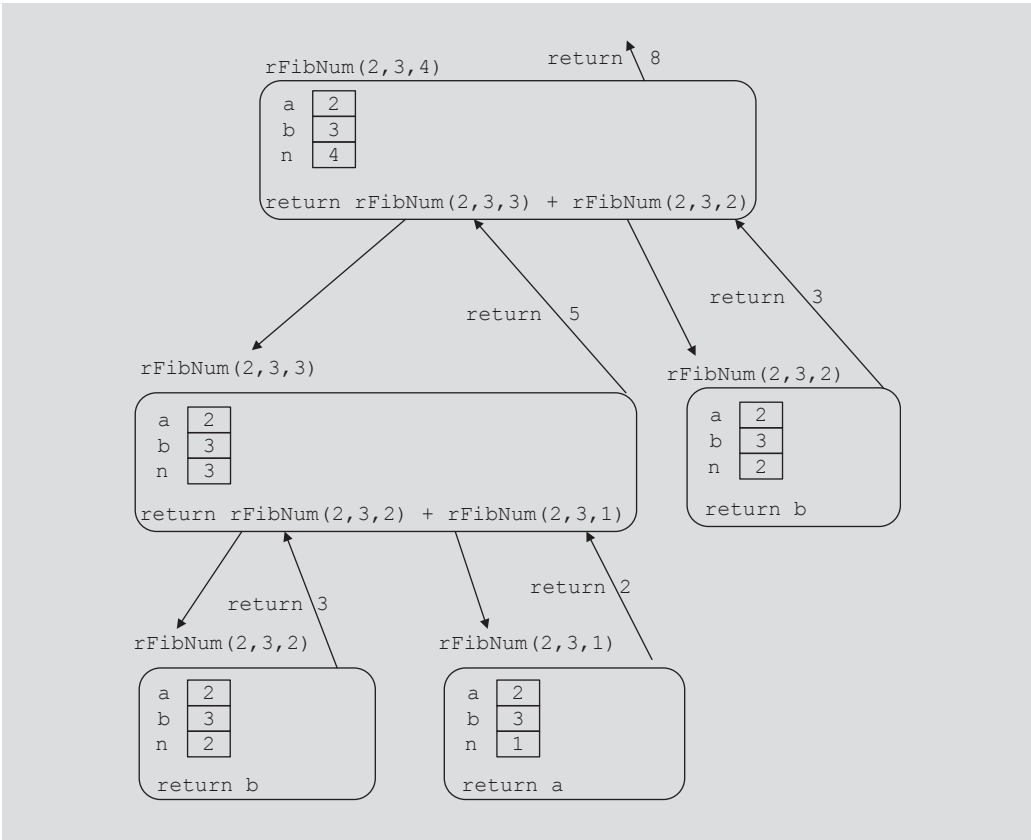


FIGURE 6-7 Execution of `rFibNum(2, 3, 4)`

Figure 6-7 reveals that the execution of the recursive version of the program to calculate a Fibonacci number is not as efficient as the execution of the nonrecursive version, even though the algorithm and the method implementing the algorithm might be simpler. In the recursive version, some values are calculated more than once. For example, to calculate `rFibNum(2, 3, 4)`, the value `rFibNum(2, 3, 2)` is calculated twice. So a recursive method might be easier to write, but might not execute as efficiently. The section "Recursion or Iteration?", presented later in this chapter, discusses these two alternatives.

The following C++ program uses the function `rFibNum`:

```
//*************************************************************
// Author: D.S. Malik
//
// Given the first two numbers of a Fibonacci sequence, this
// program uses a recursive function to determine a specific
// number(s) of a Fibonacci sequence.
//*************************************************************

#include <iostream>

using namespace std;

int rFibNum(int a, int b, int n);

int main()
{
    int firstFibNum;
    int secondFibNum;
    int nth;

    cout << "Enter the first Fibonacci number: ";
    cin >> firstFibNum;
    cout << endl;

    cout << "Enter the second Fibonacci number: ";
    cin >> secondFibNum;
    cout << endl;

    cout << "Enter the position of the desired Fibonacci number: ";
    cin >> nth;
    cout << endl;

    cout << "The Fibonacci number at position " << nth
         << " is: " << rFibNum(firstFibNum, secondFibNum, nth)
         << endl;

    return 0;
}
```

```
int rFibNum(int a, int b, int n)
{
    if (n == 1)
        return a;
    else if (n == 2)
        return b;
    else
        return rFibNum(a, b, n - 1) + rFibNum(a, b, n - 2);
}
```

Sample Run: In this sample run, the user input is shaded.

```
Enter the first Fibonacci number: 2

Enter the second Fibonacci number: 5

Enter the position of the desired Fibonacci number: 6

The Fibonacci number at position 6 is: 31
```


Tower of Hanoi

In the nineteenth century, a game called the Tower of Hanoi became popular in Europe. This game represents work that is under way in the temple of Brahma. At the creation of the universe, priests in the temple of Brahma were supposedly given three diamond needles, with one needle containing 64 golden disks. Each golden disk is slightly smaller than the disk below it. The priests' task is to move all 64 disks from the first needle to the third needle. The rules for moving the disks are as follows:

1. Only one disk can be moved at a time.
2. The removed disk must be placed on one of the needles.
3. A larger disk cannot be placed on top of a smaller disk.

The priests were told that once they had moved all the disks from the first needle to the third needle, the universe would come to an end.

Our objective is to write a program that prints the sequence of moves needed to transfer the disks from the first needle to the third needle. Figure 6-8 shows the Tower of Hanoi problem with three disks.

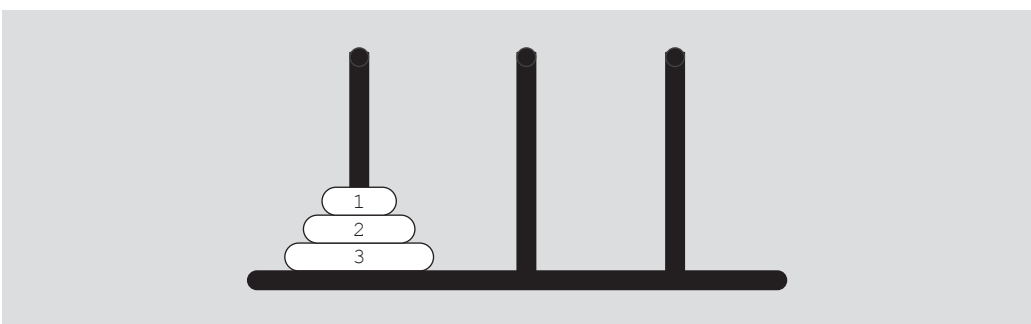


FIGURE 6-8 Tower of Hanoi problem with three disks

As before, we think in terms of recursion. Let us first consider the case when the first needle contains only one disk. In this case, the disk can be moved directly from needle 1 to needle 3. So let us consider the case when the first needle contains only two disks. In this case, first we move the first disk from needle 1 to needle 2, and then we move the second disk from needle 1 to needle 3. Finally, we move the first disk from needle 2 to needle 3. Next, we consider the case when the first needle contains three disks, and then generalize this to the case of 64 disks (in fact, to an arbitrary number of disks).

Suppose that needle 1 contains three disks. To move disk number 3 to needle 3, the top two disks must first be moved to needle 2. Disk number 3 can then be moved from needle 1 to needle 3. To move the top two disks from needle 2 to needle 3, we use the same strategy as before. This time we use needle 1 as the intermediate needle. Figure 6-9 shows a solution to the Tower of Hanoi problem with three disks.

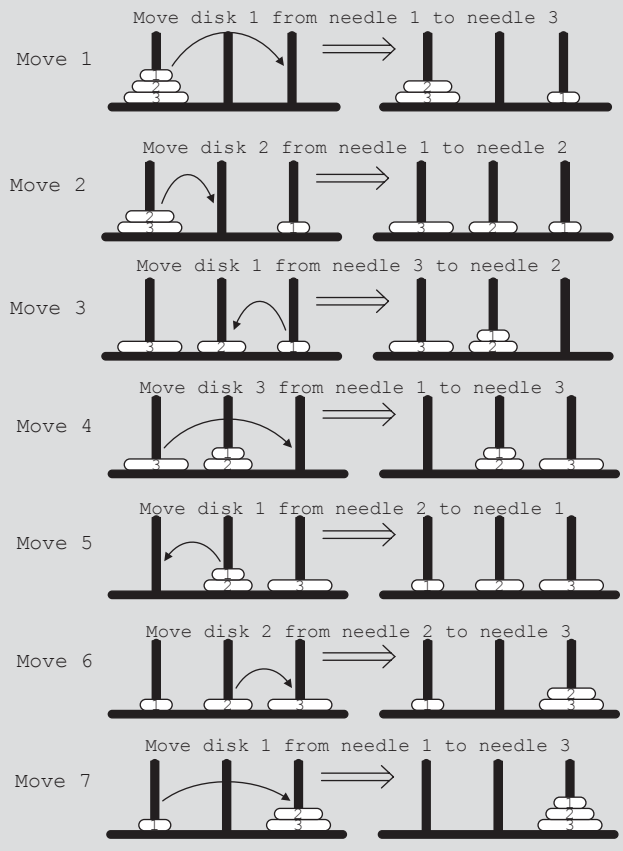


FIGURE 6-9 Solution to Tower of Hanoi problem with three disks

Let us now generalize this problem to the case of 64 disks. To begin, the first needle contains all 64 disks. Disk number 64 cannot be moved from needle 1 to needle 3 unless the top 63 disks are on the second needle. So first we move the top 63 disks from needle 1 to needle 2, and then we move disk number 64 from needle 1 to needle 3. Now the top 63 disks are all on needle 2. To move disk number 63 from needle 2 to needle 3, we first move the top 62 disks from needle 2 to needle 1, and then we move disk number 63 from needle 2 to needle 3. To move the remaining 62 disks, we use a similar procedure. This discussion translates into the following recursive algorithm given in pseudocode. Suppose that needle 1 contains n disks, where $n \geq 1$.

1. Move the top $n - 1$ disks from needle 1 to needle 2, using needle 3 as the intermediate needle.
2. Move disk number n from needle 1 to needle 3.
3. Move the top $n - 1$ disks from needle 2 to needle 3, using needle 1 as the intermediate needle.

This recursive algorithm translates into the following C++ function:

```
void moveDisks(int count, int needle1, int needle3, int needle2)
{
    if (count > 0)
    {
        moveDisks(count - 1, needle1, needle2, needle3);

        cout << "Move disk " << count << " from " << needle1
             << " to " << needle3 << "." << endl;

        moveDisks(count - 1, needle2, needle3, needle1);
    }
}
```

TOWER OF HANOI: ANALYSIS

Let us determine how long it would take to move all 64 disks from needle 1 to needle 3. If needle 1 contains 3 disks, then the number of moves required to move all 3 disks from needle 1 to needle 3 is $2^3 - 1 = 7$. Similarly, if needle 1 contains 64 disks, then the number of moves required to move all 64 disks from needle 1 to needle 3 is $2^{64} - 1$. Because $2^{10} = 1024 \approx 1000 = 10^3$, we have

$$2^{64} = 2^4 \times 2^{60} \approx 2^4 \times 10^{18} = 1.6 \times 10^{19}$$

The number of seconds in one year is approximately 3.2×10^7. Suppose the priests move one disk per second and they do not rest. Now:

$$1.6 \times 10^{19} = 5 \times 3.2 \times 10^{18} = 5 \times (3.2 \times 10^7) \times 10^{11} = (3.2 \times 10^7) \times (5 \times 10^{11})$$

The time required to move all 64 disks from needle 1 to needle 3 is roughly 5×10^{11} years. It is estimated that our universe is about 15 billion years old (1.5×10^{10}). Also,

$5 \times 10^{11} = 50 \times 10^{10} \approx 33 \times (1.5 \times 10^{10})$. This calculation shows that our universe would last about 33 times as long as it already has.

Assume that a computer can generate 1 billion (10^9) moves per second. Then the number of moves that the computer can generate in one year is:

$$(3.2 \times 10^7) \times 10^9 = 3.2 \times 10^{16}$$

So the computer time required to generate 2^{64} moves is:

$$2^{64} \approx 1.6 \times 10^{19} = 1.6 \times 10^{16} \times 10^3 = (3.2 \times 10^{16}) \times 500$$

Thus, it would take about 500 years for the computer to generate 2^{64} moves at the rate of 1 billion moves per second.

Converting a Number from Decimal to Binary

In this example, we design a program that uses recursion to convert a nonnegative integer in decimal format—that is, base 10—into the equivalent binary number—that is, base 2. First we define some terms.

Let x be an integer. We call the remainder of x after division by 2 the **rightmost bit** of x. Thus, the rightmost bit of 33 is 1 because 33 % 2 is 1, and the rightmost bit of 28 is 0 because 28 % 2 is 0. (Recall that in C++, % is the mod operator; it produces the remainder of the integer division.)

We first illustrate the algorithm to convert an integer in base 10 to the equivalent number in binary format with the help of an example.

Suppose we want to find the binary representation of 35. First, we divide 35 by 2. The quotient is 17 and the remainder—that is, the rightmost bit of 35—is 1. Next, we divide 17 by 2. The quotient is 8 and the remainder—that is, the rightmost bit of 17—is 1. Next, we divide 8 by 2. The quotient is 4 and the remainder—that is, the rightmost bit of 8—is 0. We continue this process until the quotient becomes 0.

The rightmost bit of 35 cannot be printed until we have printed the rightmost bit of 17. The rightmost bit of 17 cannot be printed until we have printed the rightmost bit of 8, and so on. Thus, the binary representation of 35 is the binary representation of 17 (that is, the quotient of 35 after division by 2), followed by the rightmost bit of 35.

Thus, to convert an integer *num* in base 10 into the equivalent binary number, we first convert the quotient *num* / 2 into an equivalent binary number, and then append the rightmost bit of *num* to the binary representation of *num* / 2.

This discussion translates into the following recursive algorithm, where `binary(num)` denotes the binary representation of `num`:

1. `binary(num) = num` if `num = 0`.
2. `binary(num) = binary(num / 2)` followed by `num % 2` if `num > 0`.

The following recursive function implements this algorithm:

```
void decToBin(int num, int base)
{
    if (num > 0)
    {
        decToBin(num / base, base);
        cout << num % base;
    }
}
```

Figure 6-10 traces the execution of the following statement:

```
decToBin(13, 2);
```

where `num` is `13` and `base` is `2`.

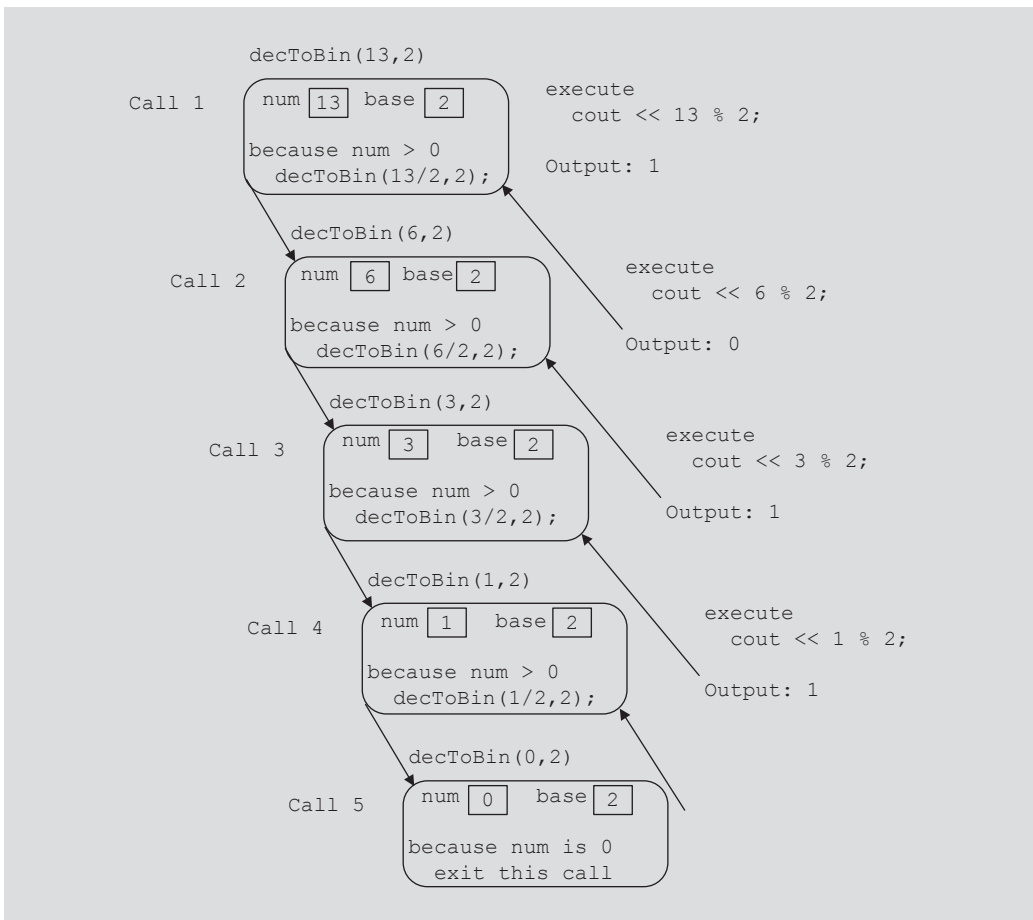


FIGURE 6-10 Execution of `decToBin(13, 2)`

Because the **if** statement in call 5 fails, this call does not print anything. The first output is produced by call 4, which prints 1; the second output is produced by call 3, which prints 1; the third output is produced by call 2, which prints 0; and the fourth output is produced by call 1, which prints 1. Thus, the output of the statement:

```
decToBin(13, 2);
```

is:

```
1101
```

The following C++ program tests the function **decToBin:**

```
//*******************************************************
// Author: D. S. Malik
//
// Program: Decimal to binary
// This program uses recursion to find the binary
// representation of a nonnegative integer.
//*******************************************************

#include <iostream>

using namespace std;

void decToBin(int num, int base);

int main()
{
    int decimalNum;
    int base;

    base = 2;

    cout << "Enter number in decimal: ";
    cin >> decimalNum;
    cout << endl;

    cout << "Decimal " << decimalNum << " = ";
    decToBin(decimalNum, base);
    cout << " binary" << endl;

    return 0;
}

void decToBin(int num, int base)
{
    if (num > 0)
    {
        decToBin(num / base, base);
        cout << num % base;
    }
}
```

Sample Run: In this sample run, the user input is shaded.

```
Enter a number in decimal: 57

Decimal 57 = 111001 binary
```

Recursion or Iteration?

Often there are two ways to solve a particular problem—recursion or iteration. The programs in the previous chapters used a loop to repeat a set of statements to perform certain calculations. In other words, the programs in the previous chapters used an iterative control structure to repeat a set of statements. Formally, **iterative control structures** use a looping structure, such as **`while`**, **`for`**, or **`do...while`**, to repeat a set of statements.

This chapter began by designing a recursive method to find the factorial of a nonnegative integer. Using an iterative control structure, we can easily write an algorithm to find the factorial of a nonnegative integer. Given our familiarity with iterative techniques, the iterative solution will seem simpler than the recursive solution. The only reason we gave a recursive solution to the factorial problem was to illustrate how recursion works using a simple example.

In this chapter, we also used recursion to determine the largest element in a list, determining a Fibonacci number. Using an iterative control structure, we can also write an algorithm to find the largest number in an array. Similarly, an algorithm that uses an iterative control structure can be designed to find the Fibonacci number.

The obvious question becomes, which approach is better? There is no general answer, but there are some guidelines. In addition to the nature of the solution, efficiency is the other key factor in determining the better approach.

When a function is called, memory space is allocated for its formal parameters and local variables. When the function terminates, that memory space is deallocated. In this chapter, while tracing the execution of recursive methods, we saw that every recursive call had its own set of parameters and local variables. That is, every recursive call required the system to allocate memory space for its formal parameters and local variables, and then deallocate the memory space when the function exited. Even though we don't need to write program statements to allocate and deallocate memory, overhead is associated with executing a recursive function, both in terms of memory space and execution time. Therefore, a recursive function executes more slowly than its iterative counterpart. On slower computers, especially those with limited memory space, the (relatively slow) execution of a recursive function is noticeable. Clearly, a recursive function is less efficient than a corresponding iterative function in terms of execution time and memory usage.

Efficiency is not determined solely by execution time and memory usage. Chances are that you *have never* been concerned with either execution time or memory usage as you have been writing C++ programs. Efficient use of a programmer's time also is an important consideration. Chances are that you *have* considered carefully what you can do to minimize the time required to produce C++ programs. Often this is entirely appropriate. As a professional programmer, your time typically is far more expensive than the cost of the

computer you use to produce programs. Of course, if you are developing software that will be used many times a day by large numbers of users, then execution time and memory usage become important considerations.

Today's computers are fast and have abundant memory. Therefore, the additional execution time and the memory consumed by a recursive function might not be noticeable. Given the ever-increasing speed and memory capacity of today's computers, the choice between iteration and recursion depends more and more on how the programmer envisions the solution to the problem, and less on execution time and memory usage. In rare instances where execution time must be minimized or the demand for memory is inordinately high, iteration might be better than recursion even when the recursive solution is more obvious. Fortunately, any program that can be written recursively also can be written iteratively.

As a general rule, if you think an iterative solution is at least as obvious and easy to construct as a recursive solution, choose the iterative solution. On the other hand, if the recursive solution is more obvious and easier to construct, such as the solution to the Towers of Hanoi problem, choose the recursive solution.

If you question the existence of problems for which a recursive solution is more obvious and easier to construct than an iterative solution, attempt to solve the Towers of Hanoi problem without using recursion. You'll quickly gain an increased appreciation for recursion. The ability to write recursive solutions is an important programming skill.

Recursion and Backtracking: 8-Queens Puzzle

This section describes a problem-solving and algorithm design technique called backtracking. Let us consider the following 8-queens puzzle: Place 8 queens on a chess-board (8 by 8 square board) so that no two queens can attack each other. For any two queens to be nonattacking, they cannot be in the same row, same column, or same diagonals. Figure 6-11 gives one possible solution to the 8-queens puzzle.

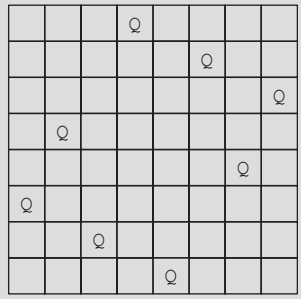


FIGURE 6-11 A solution to the 8-queens puzzle

In 1850, the 8-queens puzzle was considered by the great C. F. Gauss, who was unable to obtain a complete solution. The term backtrack was first coined by D. H. Lehmer in 1950. In 1960, R. J. Walker gave an algorithmic account of backtracking. A general description of backtracking with a variety of applications was presented by S. Golomb and L. Baumert.

Backtracking

The backtracking algorithm attempts to find solutions to a problem by constructing partial solutions and making sure that any partial solution does not violate the problem requirements. The algorithm tries to extend a partial solution toward completion. However, if it is determined that the partial solution would not lead to a solution, that is, the partial solution would end in a dead end, then the algorithm backs up by removing the most recently added part and trying other possibilities.

n-Queens Puzzle

In backtracking, the solution of the n-queens puzzle, because each queen must be placed in a different row, can be represented as an n-tuple $(x_1, x_2, \ldots, x_n)$, where x_i is an integer such that $1 \leq x_i \leq n$. In this tuple, x_i specifies the column number, where to place the ith queen in the ith row. Therefore, for the 8-queens puzzle the solution is an 8-tuple $(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)$, where x_i is the column where to place the ith queen in the ith row. For example, the solution in Figure 6-11 can be represented as the 8-tuple (4,6,8,2,7,1,3,5). That is, the first queen is placed in the first row and fourth column, the second queen is placed in the second row and sixth column, and so on. Clearly, each x_i is an integer such that $1 \leq x_i \leq 8$.

Let us again consider the 8-tuple $(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)$, where x_i is an integer such that $1 \leq x_i \leq 8$. Because each x_i has 8 choices, there are 8^8 such tuples, and so possibly 8^8 solutions. However, because no two queens can be placed in the same row, no two elements of the 8-tuple $(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)$ are the same. From this, it follows that the number of 8-tuples possibly representing solutions are 8!.

The solution that we develop can, in fact, be applied to any number of queens. Therefore, to illustrate the backtracking technique, we consider the 4-queens puzzle. That is, you are given a 4 by 4 square board (see Figure 6-12) and you are to place 4 queens on the board so that no two queens attack each other.

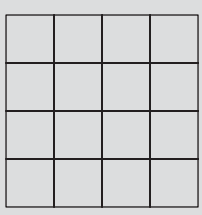


FIGURE 6-12 Square board for the 4-queens puzzle

We start by placing the first queen in the first row and first column as shown in Figure 6-13(a). (A cross in a box means no other queen can be placed in that box.)

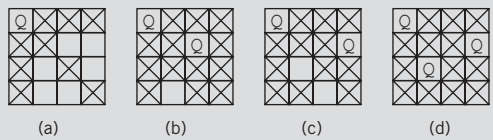


FIGURE 6-13 Finding a solution to the 4-queens puzzle

After placing the first queen, we try to place the second queen in the second row. Clearly, the first square in the second row where the second queen can be placed is the third column. So we place the second queen in that column; see Figure 6-13(b).

Next, we try to place the third queen in the third row. We find that the third queen cannot be placed in the third row and so we arrive at a dead end. At this point we backtrack to the previous board configuration and place the second queen in the fourth column; see Figure 6-13(c). Next, we try to place the third queen in the third row. This time, we successfully place the third queen in the second column of the third row; see Figure 6-13(d). After placing the third queen in the third row, when we try to place the fourth queen, we discover that the fourth queen cannot be placed in the fourth row.

We backtrack to the third row and try placing the queen in any other column. Because no other column is available for queen three, we backtrack to row two and try placing the second queen in any other column, which cannot be done. We, therefore, backtrack to the first row and place the first queen in the next column. After placing the first queen in the second column, we place the remaining queens in the successive rows. This time we obtain the solution, as shown by Figure 6-14.

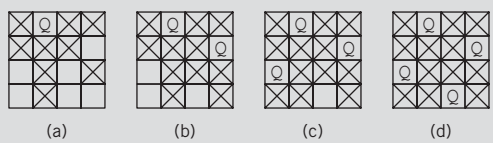


FIGURE 6-14 A solution to the 4-queens puzzle

Backtracking and the 4-Queens Puzzle

Suppose that the rows of the square board of the 4-queens puzzle are numbered 0 through 3 and the columns are numbered 0 through 3. (Recall that in C++, an array index starts at 0.)

For the 4-queens puzzle, we start by placing the first queen in the first row and first column, thus generating the tuple (0). We then place the second queen in the third column of the second row and so generate the tuple (0,2). When we try to place the third queen in the third row, we determined that the third queen cannot be placed in the third row and so we back up to the partial solution (0,2), remove 2 from the tuple and then generate the tuple (0,3), that is, the third queen is placed in the fourth column of the second row. With the partial solution (0,3), we then try to place the third queen in the third row and generate the tuple (0,3,1). Next, with the partial solution (0,3,1), when we try to place the fourth queen in the fourth row, it is determined that it cannot be done and so the partial solution (0,3,1) ends up in a dead end.

From the partial solution (0,3,1), the backtracking algorithm, in fact, backs up to placing the first queen and so removes all the elements of the tuple. The algorithm then places the first queen in the second column of the first row and thus generates the partial solution (1). In this case, the sequence of partial solution generated is, (1), (1,3), (1,3,0), and (1,3,0,2), which represents a solution to the 4-queens puzzle. The solutions generated by the backtracking algorithm can be represented by a tree, as shown in Figure 6-15.

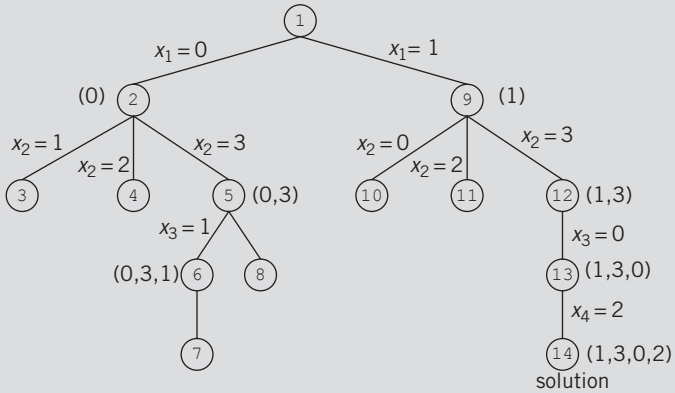


FIGURE 6-15 4-queens tree

8-Queens Puzzle

Let us now consider the 8-queens puzzle. Like the 4-queens puzzle, no two queens can be in the same row, same columns, and same diagonals. Determining whether two queens are in the same row or column is easy because we can check their row and column position. Let us describe how to determine whether two queens are in the same diagonal or not.

Consider the 8 by 8 square board shown in Figure 6-16. The rows are numbered 0 through 7; the columns are numbered 0 through 7. (Recall that, in C++, the array index starts at 0.)

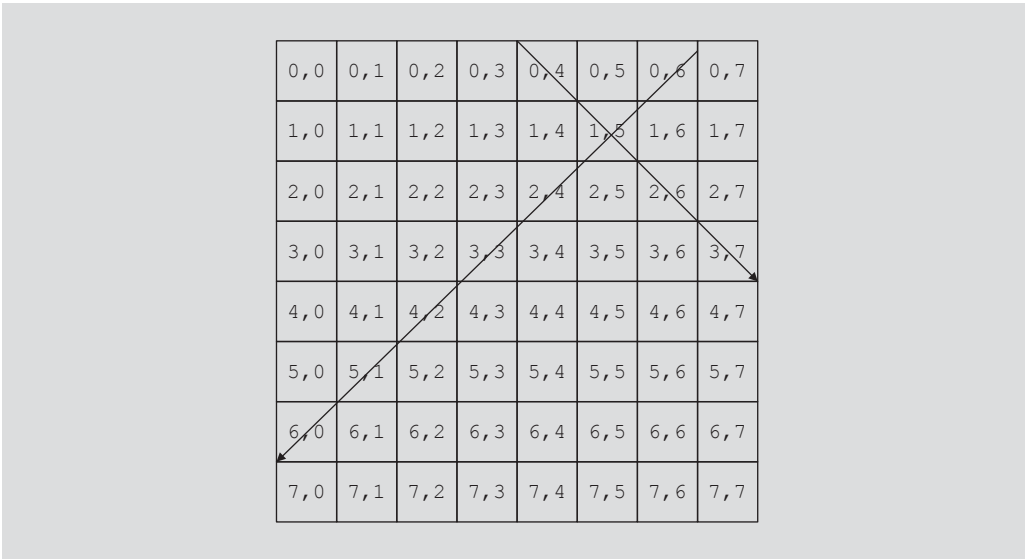


FIGURE 6-16 8 × 8 square board

Consider the diagonal from upper left to lower right, as indicated by the arrow. The positions of the squares on this diagonal are (0,4), (1,5), (2,6), and (3,7). Notice that for these entries `rowPosition` – `columnPosition` is – 4. For example, $0 - 4 = 1 - 5 = 2 - 6 = 3 - 7 = -4$. It can be shown that for each square on a diagonal from upper left to lower right, `rowPosition - columnPosition` is the same.

Now consider the diagonal from upper right to lower left as indicated by the arrow. The positions of the squares on this diagonal are (0,6), (1,5), (2,4), (3,3), (4,2), (5,1), and (6,0). Here `rowPosition + columnPosition` = 6. It can be shown that for each square on a diagonal from upper right to lower left, `rowPosition + columnPosition` is the same.

We can use these results to determine whether two queens are on the same diagonal or not. Suppose a queen is at position (i, j), (row i and column j), and another queen is at position (k, l) , (row k and column l). These queens are on the same diagonal if either $i + j = k + l$ or $i - j = k - l$. The first equation implies that $j - l = i - k$ and the second equation implies that $j - l = k - i$. From this it follows that the two queens are on the same diagonal if $|j - l| = |i - k|$, where $|j - l|$ is the absolute value of $j - l$ and so on.

Because a solution to the 8-queens puzzle is represented as an 8-tuple, we use the array `queensInRow` of size 8, where `queensInRow[k]` specifies the column position of the kth queen in row `k`. For example, `queensInRow[0] = 3` means the first queen is placed in column `3` (which is the fourth column) of row `0` (which is the first row).

Suppose that we place the first *k*-1 queens in the first *k* – 1 rows. Next we try to place the *k*th queen in a column of the *k*th row. We write the function `canPlaceQueen(k, i)`, which returns `true` if the *k*th queen can be placed in the *i*th column of row *k*; otherwise, it returns `false`.

The first *k*-1 queens are in the first *k*-1 rows and we are trying to place the *k*th queen in the *k*th row. The *k*th row, therefore, must first be empty. It thus follows that the *k*th queen can be placed in column *i* of row *k*, provided no other queen is in column *i* and no queens are on the diagonals on which square (*k*, *i*) lies. The general algorithm for the function `canPlaceQueen(k, i)` is as follows:

```
for (int j = 0; j < k; j++)
    if((queensInRow[j] == i)  //there is already a queen in column i
       || (abs(queensInRow[j] - i) == abs(j-k))) //there is already
                                  //a queen in one of the diagonals
                                  //on which square (k,i) lies
    return false;

return true;
```

The `for` loop checks whether there is already a queen either in column *i* or in one of the diagonals on which square (*k*, *i*) lies. If it finds a queen at any of such positions, the `for` loop returns the value `false`. Otherwise, the value `true` is returned.

The following class defines the *n*-queens puzzle as an ADT:

```
//*****************************************************************
// D.S. Malik
//
// This class specifies the functions to solve the n-queens puzzle.
//*****************************************************************

class nQueensPuzzle
{
public:
    nQueensPuzzle(int queens = 8);
      //constructor
      //Postcondition: noOfSolutions = 0; noOfQueens = queens;
      //   queensInRow is a pointer to the array to store the
      //   n-tuple

    bool canPlaceQueen(int k, int i);
      //Function to determine whether a queen can be placed
      //in row k and column i.
      //Postcondition: returns true if a queen can be placed in
      //    row k and column i; otherwise it returns false

    void queensConfiguration(int k);
      //Function to determine all solutions to the n-queens
      //puzzle using backtracking.
      //The function is called with the value 0.
      //Postcondition: All n-tuples representing solutions of
      //    n-queens puzzle are generated and printed.
```

```
    void printConfiguration();
      //Function to output an n-tuple containing a solution
      //to the n-queens puzzle.

    int solutionsCount();
      //Function to return the total number of solutions.
      //Postcondition: The value of noOfSolution is returned.

private:
    int noOfSolutions;
    int noOfQueens;
    int *queensInRow;
};
```

The definitions of the member functions of the `class nQueensPuzzle` are given next:

```
nQueensPuzzle::nQueensPuzzle(int queens)
{
    noOfQueens = queens;
    queensInRow = new int[noOfQueens];
    noOfSolutions = 0;
}

bool nQueensPuzzle::canPlaceQueen(int k, int i)
{
    for (int j = 0; j < k; j++)
        if ((queensInRow[j] == i)
             || (abs(queensInRow[j] - i) == abs(j-k)))
            return false;
    return true;
}
```

Using recursion, the function `queensConfiguration` implements the backtracking technique to determine all solutions to the *n*-queens puzzle. The parameter ***k*** specifies the queen to be placed in the *k*th row. Its definition is straightforward and is given next.

```
void nQueensPuzzle::queensConfiguration(int k)
{
    for (int i = 0; i < noOfQueens; i++)
    {
        if (canPlaceQueen(k, i))
        {
            queensInRow[k] = i;  //place the kth queen in column i
            if (k == noOfQueens - 1)  //all the queens are placed
                printConfiguration(); //print the n-tuple
            else
                queensConfiguration(k + 1); //determine the place
                                            //for the (k+1)th queen
        }
    }
}
```

```
void nQueensPuzzle::printConfiguration()
{
    noOfSolutions++;
    cout << "(";
    for (int i = 0; i < noOfQueens - 1; i++)
        cout << queensInRow[i] << ", ";

    cout << queensInRow[noOfQueens - 1] << ")" << endl;
}

int nQueensPuzzle::solutionsCount()
{
    return noOfSolutions;
}
```

We leave it as an exercise for you to write a program to test the *n*-queens puzzle class for various board sizes; see Programming Exercise 17 at the end of this chapter.

6

Recursion, Backtracking, and Sudoku

In the previous section, we used recursion and backtracking to solve the 8-queens problem. In this section, we introduce the well-known sudoku problem, which can be solved using recursion and backtracking. This problem involves filling numbers 1 to 9 in a partially filled, 9 × 9 grid with constraints described in this section. Figure 6-17(a) shows a partially filled grid.

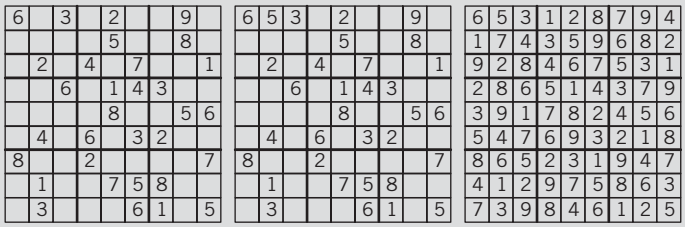


FIGURE 6-17 Sudoku problem and its solution

The sudoku grid is a 9 × 9 grid consisting of nine rows, nine columns, and nine 3 × 3 smaller grids. In Figure 6-17, the nine 3 × 3 smaller grids are separated by darker horizontal and vertical lines. The first 3 × 3 smaller grid is in rows 1 to 3 and columns 1 to 3, the second 3 × 3 smaller grid is in rows 1 to 3 and columns 4 to 6, and so on.

The objective is to fill the entire grid with numbers 1 to 9 such that each number appears exactly once in each row, each column, and each 3 × 3 smaller grid. For example, the solution to the sudoku problem in Figure 6-17(a) is shown in Figure 6-17(c).

Next, we describe, a simple recursive backtracking algorithm to find the solution to the sudoku problem in Figure 6-17(a).

Starting at the first row, we find the first empty grid slot. For example, in Figure 6-17(a), the first empty grid slot is in the second row and second column. Next we find the first number, between 1 and 9, that can be placed in this slot. (Note that before placing a number in an empty slot, we must check that the number does not appear in the row, column, and the 3×3 grid containing the slot.) For example, in Figure 6-17(a), the first number that can be placed in the second row and second column is 5; see Figure 6-17(b). After placing a number in the first empty slot, we find the next empty slot and try to place a number in that slot. If a number cannot be placed in a slot, then we must backtrack to the previous slot, where the number was placed, place a different number, and continue. If we arrive at a slot where no number can be placed, then the sudoku problem has no solutions.

The following class implements the sudoku problem as an ADT:

```
//*****************************************************************
// D.S. Malik
//
// This class specifies the functions to solve a sudoku problem.
//*****************************************************************

class sudoku
{
public:
    sudoku();
      //default constructor
      //Postcondition: grid is initialized to 0

    sudoku(int g[][9]);
      //constructor
      //Postcondition: grid = g

    void initializeSudokuGrid();
      //Function to prompt the user to specify the numbers of the
      //partially filled grid.
      //Postcondition: grid is initialized to the numbers
      //    specified by the user.

    void initializeSudokuGrid(int g[][9]);
      //Function to initialize grid to g
      //Postcondition: grid = g;

    void printSudokuGrid();
      //Function to print the sudoku grid.

    bool solveSudoku();
      //Funtion to solve the sudoku problem.
      //Postcondition: If a solution exits, it returns true,
      //    otherwise it returns false.
```

```
    bool findEmptyGridSlot(int &row, int &col);
      //Function to determine if the grid slot specified by
      //row and col is empty.
      //Postcondition: Returns true if grid[row][col] = 0;

    bool canPlaceNum(int row, int col, int num);
      //Function to determine if num can be placed in
      //grid[row][col]
      //Postcondition: Returns true if num can be placed in
      //    grid[row][col], otherwise it returns false.

    bool numAlreadyInRow(int row, int num);
      //Function to determine if num is in grid[row][]
      //Postcondition: Returns true if num is in grid[row][],
      //    otherwise it returns false.

    bool numAlreadyInCol(int col, int num);
      //Function to determine if num is in grid[row][]
      //Postcondition: Returns true if num is in grid[row][],
      //    otherwise it returns false.

    bool numAlreadyInBox(int smallGridRow, int smallGridCol,
                         int num);
      //Function to determine if num is in the small grid
      //Postcondition: Returns true if num is in small grid,
      //    otherwise it returns false.

private:
    int grid[9][9];
};
```

Note that we use the digit `0` to specify an empty grid. For example, the partially filled sudoku grid in Figure 6-17(a) is entered and stored as:

```
6 0 3 0 2 0 0 9 0
0 0 0 0 5 0 0 8 0
0 2 0 4 0 7 0 0 1
0 0 6 0 1 4 3 0 0
0 0 0 0 8 0 0 5 6
0 4 0 6 0 3 2 0 0
8 0 0 2 0 0 0 0 7
0 1 0 0 7 5 8 0 0
0 3 0 0 0 6 1 0 5
```

Next we write the definition of the function `solveSudoku`.

The function `solveSudoku` uses recursion and backtracking to find a solution, if one exists, of the partially filled sudoku grid. The general algorithm is as follows:

1. Find the position of the first empty slot in the partially filled grid. If the grid has no empty slots, return true and print the solution.

2. Suppose the variables row and col specify the position of the empty grid position.

```
          for (int digit = 1; digit <= 9; digit++)
          {
             if (grid[row][col] <> digit)
             {
                 grid[row][col] = digit;
                 recursively fill the updated grid;
                 if the grid is filled successfully, return true,
                 otherwise remove the assigned digit from grid[row][col]
                 and try another digit.
              }
            If all the digits have been tried and nothing worked, return false.
```

The definition of the function is as follows:

```
bool sudoku::solveSudoku()
{
    int row, col;

    if (findEmptyGridSlot(row, col))
    {
        for (int num = 1; num <= 9; num++)
        {
            if (canPlaceNum(row, col, num))
            {
                grid[row][col] = num;
                if (solveSudoku()) //recursive call
                    return true;
                grid[row][col] = 0;
            }
        }

        return false; //backtrack
    }
    else
        return true; //there are no empty slots
}
```

We leave it is as an exercise for you to write the definitions of the remaining functions of the `class sudoku` and program that solves sudoku problems; see Programming Exercise 18 at end of this chapter.

QUICK REVIEW

1. The process of solving a problem by reducing it to smaller versions of itself is called recursion.
2. A recursive definition defines a problem in terms of smaller versions of itself.
3. Every recursive definition has one or more base cases.
4. A recursive algorithm solves a problem by reducing it to smaller versions of itself.
5. Every recursive algorithm has one or more base cases.

6. The solution to the problem in a base case is obtained directly.
7. A function is called recursive if it calls itself.
8. Recursive algorithms are implemented using recursive functions.
9. Every recursive function must have one or more base cases.
10. The general solution breaks the problem into smaller versions of itself.
11. The general case must eventually be reduced to a base case.
12. The base case stops the recursion.
13. While tracing a recursive function:
    - Logically, you can think of a recursive function as having an unlimited number of copies of itself.
    - Every call to a recursive function—that is, every recursive call—has its own code and its own set of parameters and local variables.
    - After completing a particular recursive call, control goes back to the calling environment, which is the previous call. The current (recursive) call must execute completely before control goes back to the previous call. The execution in the previous call begins from the point immediately following the recursive call.
14. A function is called directly recursive if it calls itself.
15. A function that calls another function and eventually results in the original function call is said to be indirectly recursive.
16. A recursive function in which the last statement executed is the recursive call is called a tail recursive function.
17. To design a recursive function, you must do the following:
    a. Understand the problem requirements.
    b. Determine the limiting conditions. For example, for a list the limiting condition is the number of elements in the list.
    c. Identify the base cases and provide a direct solution to each base case.
    d. Identify the general cases and provide a solution to each general case in terms of smaller versions of itself.

EXERCISES

1. Mark the following statements as true or false.
    a. Every recursive definition must have one or more base cases.
    b. Every recursive function must have one or more base cases.
    c. The general case stops the recursion.
    d. In the general case, the solution to the problem is obtained directly.
    e. A recursive function always returns a value.

2. What is a base case?
3. What is a recursive case?
4. What is direct recursion?
5. What is indirect recursion?
6. What is tail recursion?
7. Consider the following recursive function:

```
int mystery(int number)                          //Line 1
{                                                //Line 2
    if (number == 0)                             //Line 3
        return number;                           //Line 4
    else                                         //Line 5
       return(number + mystery(number - 1));     //Line 6
}                                                //Line 7
```

   a. Identify the base case.
   b. Identify the general case.
   c. What valid values can be passed as parameters to the function `mystery`?
   d. If `mystery(0)` is a valid call, what is its value? If not, explain why.
   e. If `mystery(5)` is a valid call, what is its value? If not, explain why.
   f. If `mystery(-3)` is a valid call, what is its value? If not, explain why.

8. Consider the following recursive function:

```
void funcRec(int u, char v)                  //Line 1
{                                            //Line 2
   if (u == 0)                               //Line 3
      cout << v;                             //Line 4
   else if (u == 1)                          //Line 5
       cout << static_cast<char>
               (static_cast<int>(v) + 1);    //Line 6
   else                                      //Line 7
      funcRec(u - 1, v);                     //Line 8
}                                            //Line 9
```

   Answer the following questions:

   a. Identify the base case.
   b. Identify the general case.
   c. What is the output of the following statement?

```
funcRec(5, 'A');
```

9. Consider the following recursive function:

```
void exercise(int x)
{
    if (x > 0 && x < 10)
    {
        cout << x << " ";
        exercise(x + 1);
    }
}
```

What is the output of the following statements?

a. `exercise(0);`

b. `exercise(5);`

c. `exercise(10);`

d. `exercise(-5);`

10. Consider the following function:

```
int test(int x, int y)
{
    if (x == y)
        return x;
    else if (x > y)
         return (x + y);
    else
        return test(x + 1, y - 1);
}
```

What is the output of the following statements?

a. `cout << test(5, 10) << endl;`

b. `cout << test(3, 9) << endl;`

11. Consider the following function:

```
int func(int x)
{
    if (x == 0)
        return 2;
    else if (x == 1)
        return 3;
    else
        return (func(x - 1) + func(x - 2));
}
```

What is the output of the following statements?

a. `cout << func(0) << endl;`

b. `cout << func(1) << endl;`

c. `cout << func(2) << endl;`

d. `cout << func(5) << endl;`

12. Suppose that `intArray` is an array of integers, and `length` specifies the number of elements in `intArray`. Also, suppose that `low` and `high` are two integers such that `0 <= low < length`, `0 <= high < length`, and `low < high`. That is, `low` and `high` are two indices in `intArray`. Write a recursive definition that reverses the elements in `intArray` between `low` and `high`.

13. Write a recursive algorithm to multiply two positive integers *m* and *n* using repeated addition. Specify the base case and the recursive case.

14. Consider the following problem: How many ways can a committee of four people be selected from a group of 10 people? There are many other similar problems, where you are asked to find the number of ways to select a set of items from a given set of items. The general problem can be stated as follows: Find the number of ways r different things can be chosen from a set of n items, where r and n are nonnegative integers and $r \leq n$. Suppose $C(n, r)$ denotes the number of ways r different things can be chosen from a set of n items. Then $C(n, r)$ is given by the following formula:

$$C(n, r) = \frac{n!}{r!(n - r)!}$$

where the exclamation point denotes the factorial function. Moreover, $C(n, 0) = C(n, n) = 1$. It is also known that $C(n, r) = C(n - 1, r - 1) + C(n - 1, r)$.

   a. Write a recursive algorithm to determine $C(n, r)$. Identify the base case(s) and the general case(s).
   b. Using your recursive algorithm, determine $C(5, 3)$ and $C(9, 4)$.

PROGRAMMING EXERCISES

1. Write a recursive function that takes as a parameter a nonnegative integer and generates the following pattern of stars. If the nonnegative integer is 4, the pattern generated is as follows:

```
****
***
**
*
*
**
***
****
```

   Also, write a program that prompts the user to enter the number of lines in the pattern and uses the recursive function to generate the pattern. For example, specifying 4 as the number of lines generates the preceding pattern.

2. Write a recursive function to generate a pattern of stars such as the following:

```
*
**
***
****
****
***
**
*
```

Also, write a program that prompts the user to enter the number of lines in the pattern and uses the recursive function to generate the pattern. For example, specifying 4 as the number of lines generates the preceding pattern.

3. Write a recursive a function to generate the following pattern of stars:

```
   *
  * *
 * * *
* * * *
 * * *
  * *
   *
```

Also, write a program that prompts the user to enter the number of lines in the pattern and uses the recursive function to generate the pattern. For example, specifying 4 as the number of lines generates the preceding pattern.

4. Write a recursive function, `vowels`, that returns the number of vowels in a string. Also, write a program to test your function.
5. Write a recursive function that finds and returns the sum of the elements of an `int` array. Also, write a program to test your function.
6. A palindrome is a string that reads the same both forward and backward. For example, the string `"madam"` is a palindrome. Write a program that uses a recursive function to check whether a string is a palindrome. Your program must contain a value-returning recursive function that returns `true` if the string is a palindrome and `false` otherwise. Do not use any global variables; use the appropriate parameters.
7. Write a program that uses a recursive function to print a string backward. Do not use any global variables; use the appropriate parameters.
8. Write a recursive function, `reverseDigits`, that takes an integer as a parameter and returns the number with the digits reversed. Also, write a program to test your function.
9. Write a recursive function, `power`, that takes as parameters two integers x and y such that x is nonzero and returns x^y. You can use the following recursive definition to calculate x^y. If $y \geq 0$,

$$power(x, y) = \begin{cases} 1 & \text{if} \quad y = 0 \\ x & \text{if} \quad y = 1 \\ x \times power(x, y - 1) & \text{if} \quad y > 1. \end{cases}$$

If $y < 0$,

$$power(x, y) = \frac{1}{power(x, -y)}.$$

Also, write a program to test your function.

10. **(Greatest Common Divisor)** Given two integers x and y, the following recursive definition determines the greatest common divisor of x and y, written $\gcd(x,y)$:

$$\gcd(x, y) = \begin{cases} x & \text{if} \quad y = 0 \\ \gcd(y, x\%y) & \text{if} \quad y \neq 0 \end{cases}$$

Note: In this definition, % is the mod operator.

Write a recursive function, `gcd`, that takes as parameters two integers and returns the greatest common divisor of the numbers. Also, write a program to test your function.

11. Write a recursive function to implement the recursive algorithm of Exercise 12 (reversing the elements of an array between two indices). Also, write a program to test your function.

12. Write a recursive function to implement the recursive algorithm of Exercise 13 (multiplying two positive integers using repeated addition). Also, write a program to test your function.

13. Write a recursive function to implement the recursive algorithm of Exercise 14 (determining the number of ways to select a set of things from a given set of things). Also, write a program to test your function.

14. In the section "Converting a Number from Decimal to Binary," in this chapter, you learned how to convert a decimal number into the equivalent binary number. Two more number systems, octal (base 8) and hexadecimal (base 16), are of interest to computer scientists. In fact, in C++, you can instruct the computer to store a number in octal or hexadecimal.

The digits in the octal number system are 0, 1, 2, 3, 4, 5, 6, and 7. The digits in the hexadecimal number system are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F. So A in hexadecimal is 10 in decimal, B in hexadecimal is 11 in decimal, and so on.

The algorithm to convert a positive decimal number into an equivalent number in octal (or hexadecimal) is the same as discussed for binary numbers. Here, we divide the decimal number by 8 (for octal) and by 16 (for hexadecimal). Suppose a_b represents the number a to the base b. For example, 75_{10} means 75 to the base 10 (that is decimal), and 83_{16} means 83 to the base 16 (that is, hexadecimal). Then:

$753_{10} = 1361_8$
$753_{10} = 2F1_{16}$

The method of converting a decimal number to base 2, or 8, or 16 can be extended to any arbitrary base. Suppose you want to convert a decimal number n into an equivalent number in base b, where b is between 2 and 36. You then divide the decimal number n by b as in the algorithm for converting decimal to binary.

Note that the digits in, say base 20, are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, and J.

Write a program that uses a recursive function to convert a number in decimal to a given base *b*, where *b* is between 2 and 36. Your program should prompt the user to enter the number in decimal and in the desired base.

Test your program on the following data:

9098 and base 20

692 and base 2

753 and base 16

15. **(Recursive Sequential Search)** The sequential search algorithm given in Chapter 3 is nonrecursive. Write and implement a recursive version of the sequential search algorithm.

16. The function `sqrt` from the header file `cmath` can be used to find the square root of a nonnegative real number. Using Newton's method, you can also write an algorithm to find the square root of a nonnegative real number within a given tolerance as follows: Suppose x is a nonnegative real number, a is the approximate square root of x, and *epsilon* is the tolerance. Start with $a = x$;

    a. If $|a^2 - x| \leq epsilon$, then a is the square root of x within the tolerance; otherwise:

    b. Replace a with $(a^2 + x) / (2a)$ and repeat Step a

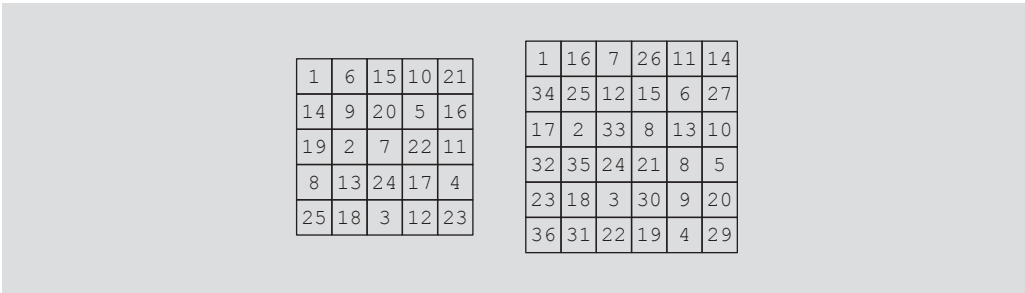
    where $|a^2 - x|$ denotes the absolute value of $a^2 - x$.

    Write a recursive function to implement this algorithm to find the square root of a nonnegative real number. Also, write a program to test your function.

17. Write a program to find solutions to the n-queens puzzle for various values of n. To be specific, test your program for $n = 4$ and $n = 8$.

18. Write the definitions of the remaining functions of the `class sudoku`. Also write a program to solve some sudoku problems.

19. **(Knight's Tour)** This chapter described the backtracking algorithm and how to use recursion to implement it. Another well-known chessboard problem that can be solved using the backtracking algorithm is a knight's tour. Given an initial board position, determine a sequence of moves by a knight that visits every square of the chessboard exactly once. For example, for a 5×5 and 6×6 square board, the sequence of moves are shown in Figure 6-18:



1	6	15	10	21
14	9	20	5	16
19	2	7	22	11
8	13	24	17	4
25	18	3	12	23

1	16	7	26	11	14
34	25	12	15	6	27
17	2	33	8	13	10
32	35	24	21	8	5
23	18	3	30	9	20
36	31	22	19	4	29

FIGURE 6-18 Knight's tour

A knight moves by jumping two positions either vertically or horizontally and one position in the perpendicular direction. Write a recursive backtracking program that takes as input an initial board position and determines a sequence of moves by a knight that visits each square of the board exactly once.